

深圳大学考试答题纸

(以论文、报告等形式考核专用)

二〇二五 ~ 二〇二六 学年度第 一 学期

课程编号 1504650001 课序号 02 课程名称 计算机视觉 主讲教师 罗笑玲 评分

学 号 2022152021 姓名 王培鸿 专业年级 计算机科学与技术（卓越班）

教师评语：

题目： 基于深度学习的肺炎感染影像智能识别方法与应用

说明：

不要删除或修改蓝色标记的文字，也不要删除线框。

请在相应的线框内答题，答题时请用 5 号、宋体、黑色文字、行距 1.25 倍。

摘要

肺炎是一种常见的呼吸系统疾病，在临床诊断中通常依赖胸部 CT 影像进行判断。然而，人工阅片方式不仅工作量大，而且诊断结果容易受到医生经验影响。随着深度学习技术的发展，卷积神经网络（Convolutional Neural Network, CNN）在图像分类领域取得了显著成果[1]，为医学影像的自动分析提供了新的解决思路。

本文以肺炎 CT 影像为研究对象，基于 AlexNet、GoogLeNet 和 VGG19 三种经典卷积神经网络模型，构建肺炎感染影像的二分类识别系统。通过引入迁移学习方法，对比分析是否使用 ImageNet 预训练权重、不同学习率以及不同训练轮数对模型性能的影响。实验结果表明，使用预训练权重能够有效提升模型的分类准确率，其中 GoogLeNet 模型在整体性能上表现最好。最后，基于实验中性能最优的模型实现了一个简单的肺炎感染影像智能识别应用界面，可视化模型分类结果。

关键词（3-5 个）：肺炎影像分类；深度学习；卷积神经网络；迁移学习；医学影像分析

第一节 引言

1.1 研究背景与意义

肺炎是指由细菌、病毒等病原体引起的肺部炎症性疾病，若不能及时诊断和治疗，可能对患者生命健康造成严重威胁。在临床实践中，胸部 CT 影像能够直观反映肺部病灶情况，是肺炎诊断的重要依据。然而，由于 CT 影像数量大、结构复杂，完全依靠人工阅片不仅效率较低，而且容易出现误诊或漏诊问题。

近年来，人工智能技术迅速发展，深度学习已成为计算机视觉领域的重要研究方向。卷积神经网络如 GoogLeNet、VGG、AlexNet，能够自动从图像中学习多层次特征，在大规模图像分类任务中取得了优于传统方法的效果[1, 2]。因此，将深度学习方法应用于肺炎影像分析，有助于提高诊断效率和准确性，对临床辅助诊断具有积极意义。

1.2 研究目标与主要工作

本文以肺炎感染影像智能识别为研究目标，主要完成以下工作：

- 1) 构建基于深度学习的肺炎影像二分类整体流程；
- 2) 选用 AlexNet、GoogLeNet 和 VGG19 三种经典 CNN 模型进行实验；
- 3) 对比分析不同预训练策略、学习率和训练轮数对模型性能的影响；
- 4) 基于性能最优模型，设计并实现简单的肺炎感染影像识别应用。

第二节 相关工作

2.1 卷积神经网络的发展

卷积神经网络最早由 LeCun 等人提出，并在手写数字识别任务中取得成功[4]。2012 年，AlexNet 在 ImageNet 图像分类竞赛中取得突破性成绩，使深度学习方法受到广泛关注[1]。随后，研究者提出了更深层的网络结构，如 VGG 网络，通过增加网络深度提升特征表达能力[2]。GoogLeNet 则引入 Inception 结构，在保证性能的同时减少了模型参数数量[3]。

2.2 深度学习在医学影像中的应用

随着计算能力的提升，深度学习逐渐被应用于医学影像分析领域。研究表明，CNN 在医学图像分类、病灶检测等任务中具有良好表现[5]。在肺炎影像识别方面，已有学者利用深度学习模型在胸部 X 光或 CT 影像中实现肺炎自动检测，并取得较高的识别准确率[6]。此外，迁移学习被广泛用于解决医学影像数据规模较小的问题[7]。

第三节 方法设计

3.1 系统整体流程

本文设计的肺炎感染影像智能识别系统主要包括以下几个步骤：

- 1) 数据预处理：对原始 CT 影像进行尺寸统一、归一化处理；
- 2) 特征提取：利用 CNN 模型自动学习影像特征；
- 3) 分类预测：通过全连接层完成肺炎阳性和阴性的分类；
- 4) 结果输出：输出预测结果并用于性能评估。

3.2 模型选择与结构调整

本文选用 AlexNet、GoogLeNet 和 VGG19 三种经典卷积神经网络模型作为特征提取网络。为适应肺炎二分类任务。

3.2.1 AlexNet 模型

AlexNet 输入为 RGB 三通道的 $224 \times 224 \times 3$ 大小的图像（也可填充为 $227 \times 227 \times 3$ ）。AlexNet 共包含 5 个卷积层（包含 3 个池化）和 3 个全连接层。其中，每个卷积层都包含卷积核、偏置项、ReLU 激活函数和局部响应归一化（LRN）模块。第 1、2、5 个卷积层后面都跟着一个最大池化层，后三个层为全连接层。最终输出层为 softmax，将网络输出转化为概率值，用于预测图像类别。

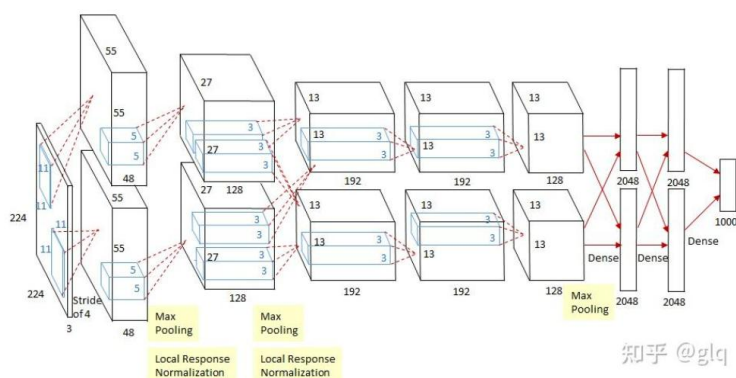


图 1 AlexNet 网络结构图

3.2.2 GoogLeNet 模型

GoogLeNet(又称 Inception v1)是 2014 年 ImageNet 竞赛的冠军模型,核心创新是 **Inception 模块**,通过并行多尺度卷积与池化的融合,在提升模型感受野多样性的同时,大幅降低参数量和计算量,实现了精度与效率的平衡。网络开头是 1 个 7×7 卷积核(步幅 2) + 最大池化(3×3 , 步幅 2) + 局部响应归一化(LRN);在网络中间的两个 Inception 模块之后,各设置 1 个小型辅助分类器。GoogLeNet 共有 22 层(含卷积、池化层),参数量仅约 500 万,远少于 AlexNet 的 6000 万。

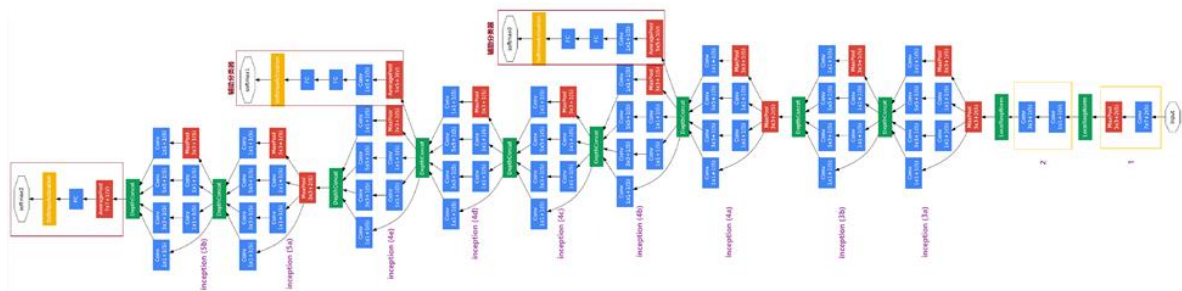


图 2 GoogLeNet 网络结构图

3.2.3 VGG19 模型

VGG19 包含了 19 个隐藏层(16 个卷积层和 3 个全连接层),所有卷积层有相同的配置,即卷积核大小为 3×3 ,步长为 1,填充为 1;共有 5 个最大池化层,大小都为 2×2 ,步长为 2;共有三个全连接层,前两层都有 4096 通道,第三层共 1000 路及代表 1000 个标签类别;最后一层为 softmax 层;所有隐藏层后都带有 ReLU 非线性激活函数。

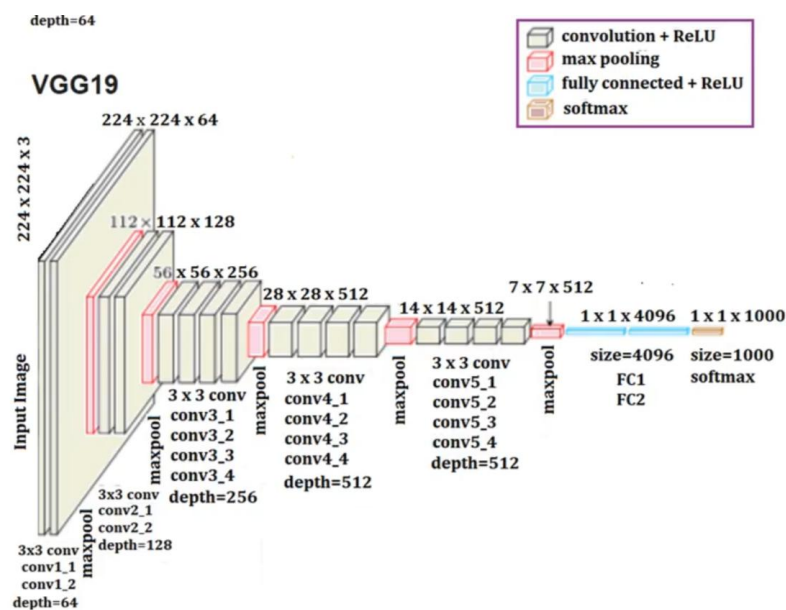


图 3 VGG19 网络结构图

3.3 训练策略

模型训练采用交叉熵损失函数,优化器选用随机梯度下降(SGD)。通过设置不同学习率和训练轮数,比较模型在不同参数设置下的性能表现。同时,采用迁移学习方法,仅对高层网络参数进行微调,以提高训练效率和模型泛化能力[7]。代码实现如下图所示。

```

criterion = nn.CrossEntropyLoss().to(device)
optimizer = optim.SGD(model.parameters(), lr=lr, weight_decay=wd)

--

```

图 4 交叉熵和 SGD 实现

3.3.1 交叉熵损失

交叉熵损失主要用于分类任务中，特别是：二分类任务：如判断图像是否包含目标物体。多分类任务：如文本分类、情感分析、手写数字识别等。如下图所示，真实标签 y 通常取值为 0 或 1。预测值 $\hat{y}$ 是模型输出的类别 1 的概率。

$$L_{BCE} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

图 5 交叉熵损失计算

3.3.2 随机梯度下降

SGD 的核心思想是在每次迭代中随机选择一个样本（或一小批样本）来估计梯度，而不是使用整个数据集。这样做的优点是计算效率高，尤其是当数据集很大时。SGD 也能够逃离局部最小值，因为随机性引入了一定的噪声，有助于模型探索更多的参数空间。如下图所示，在每次迭代中，我们计算当前参数 θ 下的梯度，然后沿梯度的反方向更新参数，以减小损失函数的值。

$$\theta_{t+1} = \theta_t - \eta \nabla f_i(\theta_t)$$

图 6 SGD 公式图

第四节 实验设置

4.1 数据集与预处理

实验使用肺炎 CT 影像数据集，其中包含 349 张肺炎阳性图像和 397 张肺炎阴性图像。数据集按照 9:1 的比例划分为训练集和测试集。数据集通过 DataSet 类进行加载，返回的数据为 RGB 格式的图片 and 分类标签，代码如下所示。

```

45 class DataSet(Dataset):
46     def __init__(self, data, cls2idx, transform):
47         self.data = data
48         self.cls2idx = cls2idx
49         self.transform = transform
50
51     def __len__(self):
52         return len(self.data)
53
54     def __getitem__(self, item):
55         image = Image.open(self.data[item][0]).convert('RGB')
56         image = self.transform(image)
57         cls = self.cls2idx[self.data[item][1]]
58         return image, cls

```

图 7 DataSet 类实现

本文对训练集和验证集的图像进行图像大小统一、归一化的操作；同时，为减少过拟合现象，对训练数据集进行了随机水平翻转的数据增强操作[8]，最后通过 `DataLoader` 类加载数据集。代码如下所示。

```
train_set = DataSet(train_data, cls2idx, transform=transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
]))
val_set = DataSet(val_data, cls2idx, transform=transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
]))

train_loader = DataLoader(train_set, batch_size=batch_size,
                           shuffle=True, num_workers=num_workers)
val_loader = DataLoader(val_set, batch_size=batch_size,
                        shuffle=True, num_workers=num_workers)
```

图 8 数据集处理

4.2 评估指标

本文主要采用以下指标评估模型性能：准确率（Accuracy）、混淆矩阵（Confusion Matrix）。

4.2.1 混淆矩阵

混淆矩阵是一个 $n \times n$ 的矩阵， n 代表的是分类标签个数。对于二分类任务，预测值与真实值都为 1 或 0，便有四种组合，如下图所示。

其中，TP，又称为 True Positive：当模型预测值和真实值均为 1；TN，又称为 True Negative 性：当模型预测值和真实值均为 0；

FP，又称为 False Positive：当模型预测为 1，而真实值为 0；FN，又称为 False Negative：当模型预测为 0，而真实值为 1。

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

图 9 混淆矩阵图

4.3 实验环境

实验在 Python 环境下完成，使用 PyTorch 深度学习框架，训练过程在 GPU 环境中进行。

第五节 实验结果与分析

5.1 AlexNet 模型

本文基于 AlexNet 模型，对比预训练与非预训练模型、不同学习率和训练轮次的训练效果。

5.1.1 预训练与非预训练模型对比

固定训练轮次为 70，在学习率为 $1e-3$ 、 $1e-4$ 、 $3e-4$ 中分别对预训练与非预训练模型进行训练，分别选取出效果最好的模型进行对比。

● 预训练模型

固定训练轮次为 70，在学习率为 $1e-3$ 、 $1e-4$ 、 $3e-4$ 中，对预训练模型进行迁移学习，结果如图所示。

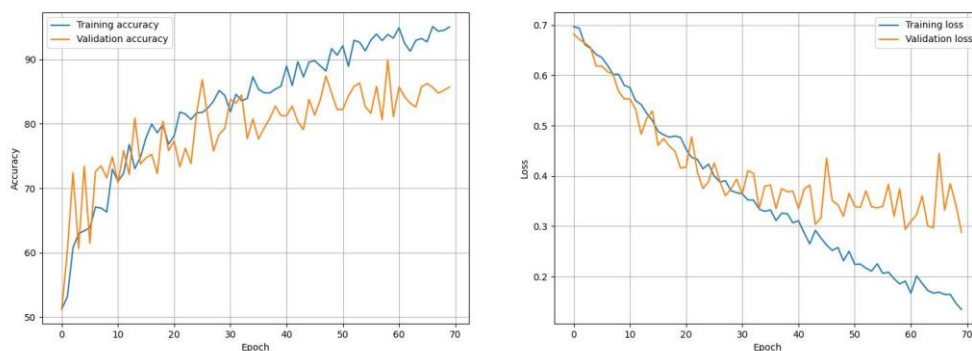


图 10 学习率为 $1e-3$

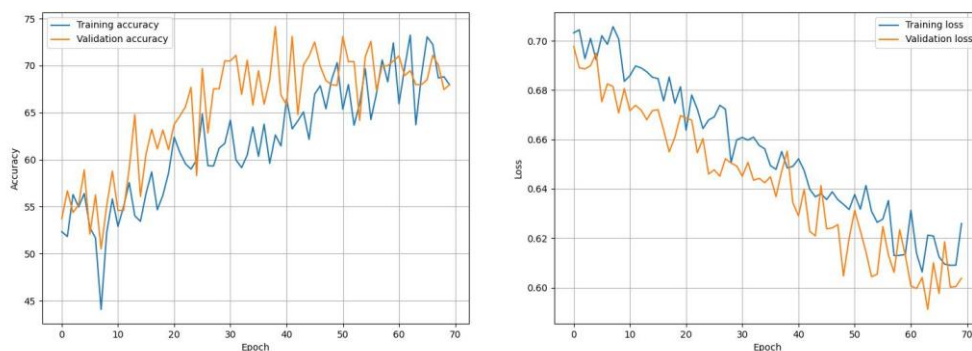


图 11 学习率为 $1e-4$

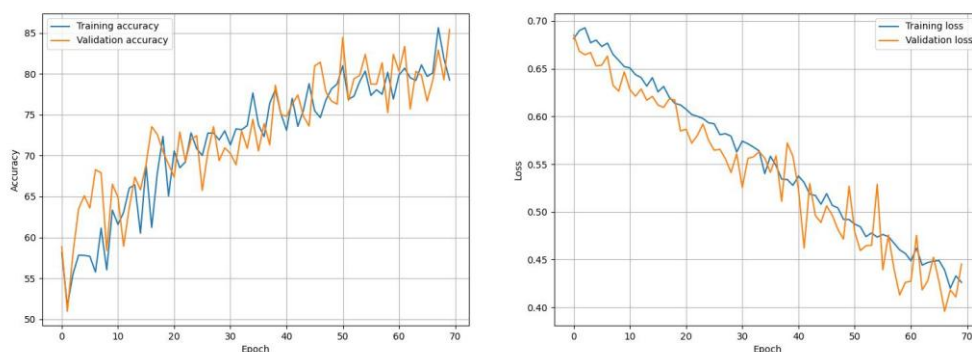


图 12 学习率为 $3e-4$

实验结果表明，在不同学习率的情况下，模型都逐渐地走向收敛，其中在学习率为 $1e-3$ 时模型准确率达到 90% 以上，但是从损失函数看出出现过拟合现象；而其他学习率下模型地准确率随着训练步数的增加而不断提高。

● 非预训练模型

固定训练轮次为 70，在学习率为 $1e-3$ 、 $1e-4$ 、 $3e-4$ 中，对非预训练模型进行从 0 训练模型，结果如图所示。

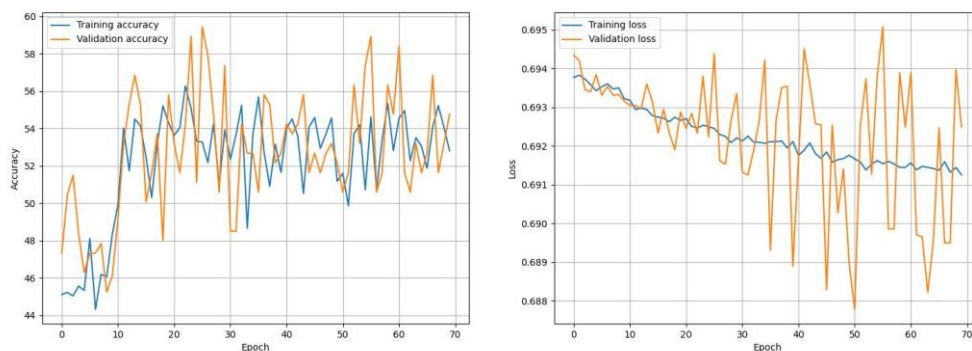


图 13 学习率为 $1e-3$

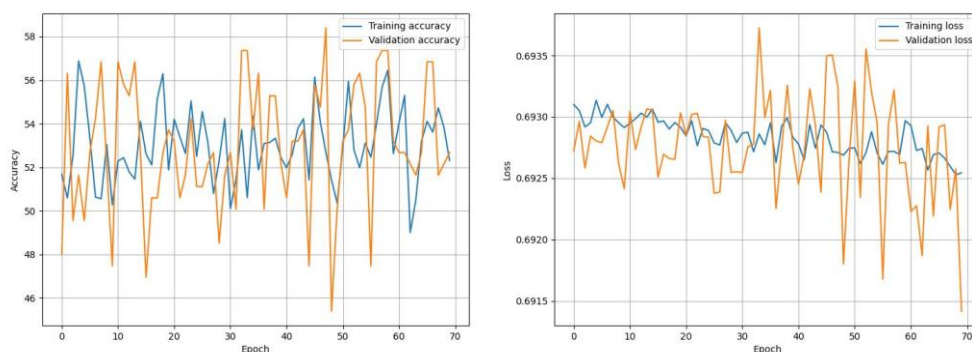


图 14 学习率为 $1e-4$

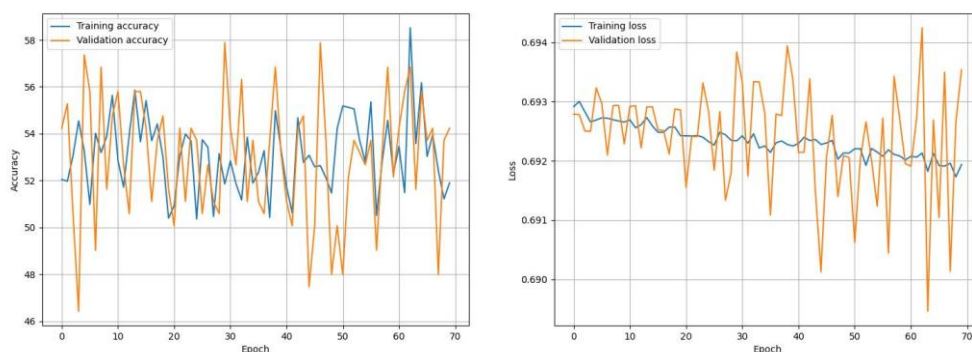


图 15 学习率为 $3e-4$

实验结果表明，在不同学习率的情况下，模型收敛缓慢，且训练损失下降及其缓慢，验证损失震荡激烈；其中在学习率为 $1e-3$ 时模型损失下降最快，但所有学习率下训练的模型最终准确率均在 60% 以下。

● 结论

以上实验结果表明，使用 ImageNet 预训练权重的模型在测试集上的准确率明显高于未使用预训练权重的模型。这说明迁移学习能够有效提升模型的特征提取能力，加快模型收敛速度[7]。故本文往后实验均基于在 ImageNet 的预训练权重进行训练。

5.1.2 不同学习率与训练轮数对比

● 不同训练轮次

固定学习率为 $1e-3$ ，采用预训练模型进行训练，对比不同训练轮次对模型效果的影响。如下图所示。

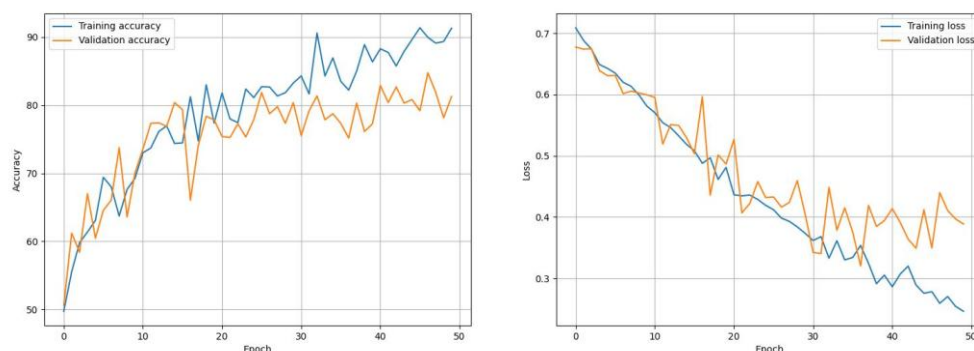


图 16 轮次为 50

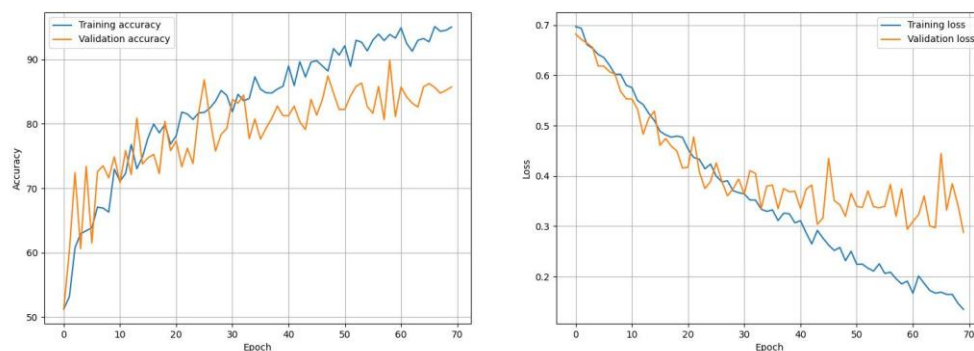


图 17 轮次为 70

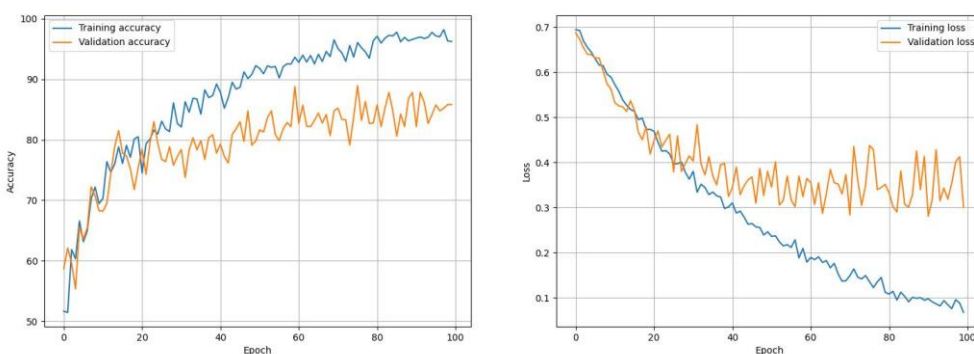


图 18 轮次为 100

以上结果表明，训练轮次对模型性能具有显著影响。随着训练轮次增加，模型在训练集上的准确率持续提升，但在验证集上的性能提升逐渐趋于饱和。当训练轮次达到 70 左右时，模型在验证集上取得了较优且稳定的性能；继续增加训练轮次至 100 时，训练集与验证集性能差距扩大，出现明显过拟合现象。因此，本文最终选取 70 轮作为模型的训练轮次。

● 不同学习率

在预训练模型的基础上，固定训练轮次为 70，比较学习率为 $1e-3$ 、 $1e-4$ 、 $3e-4$ 对模型训练效果的影响。训练结果如下图所示。

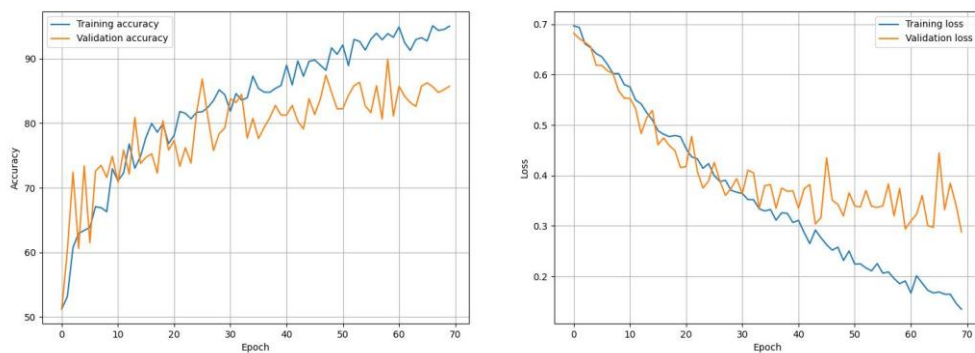


图 19 学习率为 $1e-3$ 结果图

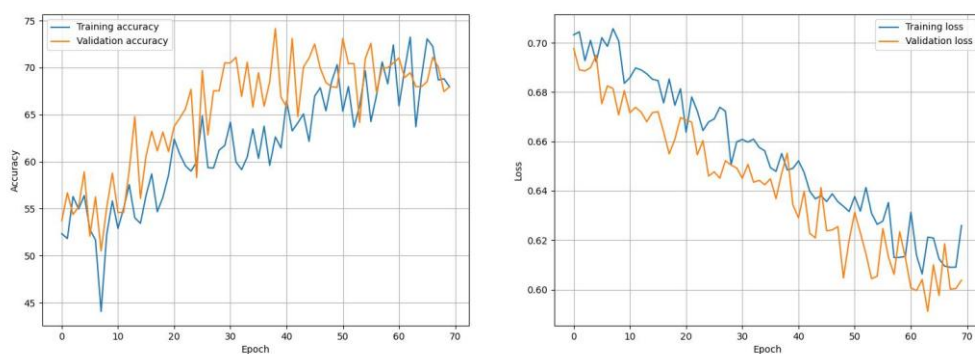


图 20 学习率为 $1e-4$ 结果图

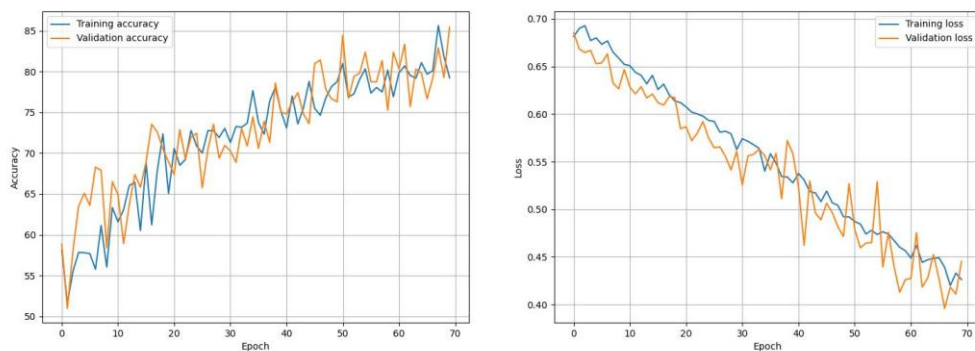


图 21 学习率为 $3e-4$ 结果图

不同学习率对模型训练过程具有显著影响。实验结果表明，较大的学习率（ $1e-3$ ）虽然能够加快模型收敛，但容易在训练后期产生过拟合；较小的学习率（ $1e-4$ ）则导致模型收敛缓慢、性能不足，最终的训练集准确率只有 75% 左右。相比之下，学习率为 $3e-4$ 时模型在收敛速度与泛化性能之间取得了较好的平衡，最终的训练集准确率只有 80% 左右，因此本文选取 $3e-4$ 作为最终训练的学习率。

● 结论

以上结果显示：过大的学习率容易导致模型不收敛，而过小的学习率则会使训练过程过慢。适当的训练轮数能够在保证模型性能的同时避免过拟合。

5.1.3 最终模型选取

综上考虑，最终选取基于预训练权重、学习率为 $3e-4$ 、训练轮次为 70 的模型作为最终的模型，训练的指标如下所示。

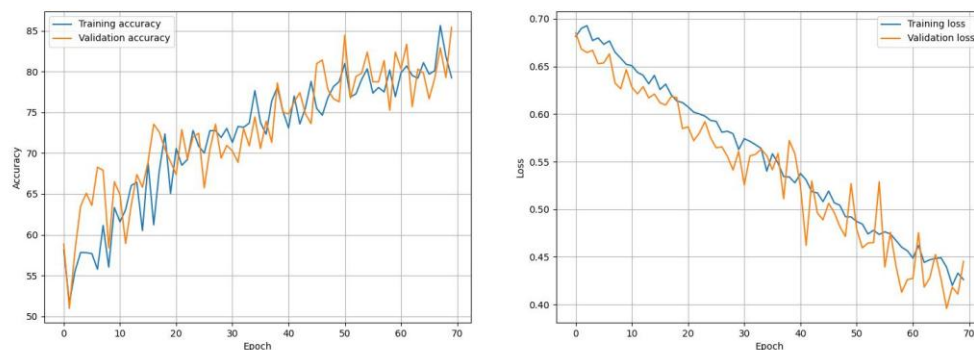


图 22 轮次为 70、学习率为 $3e-4$

5.2 GoogLeNet 模型

对于 GoogLeNet 网络，本文采用基于预训练模型权重，固定训练轮次为 70，比较不同学习率的方式进行模型训练。训练结果如图所示。

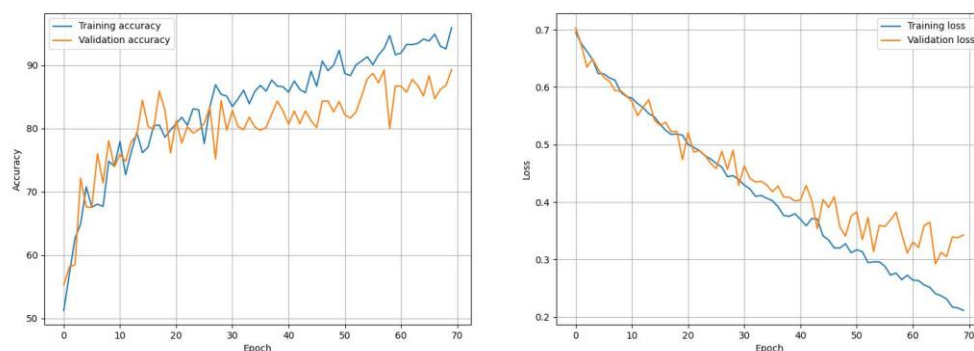


图 23 学习率为 $1e-3$

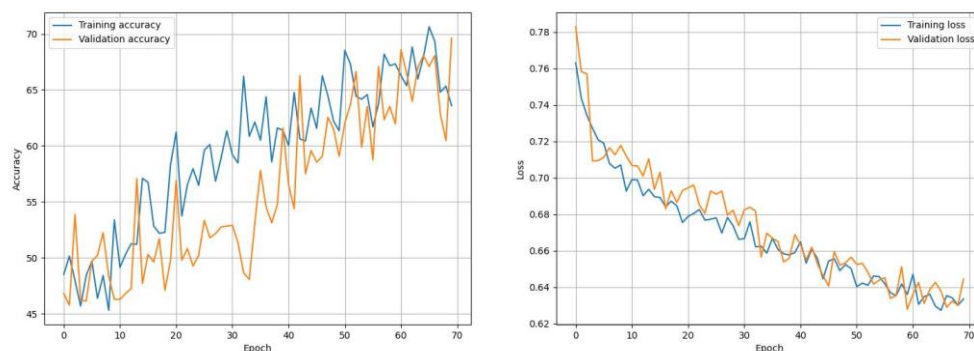


图 24 学习率为 $1e-4$

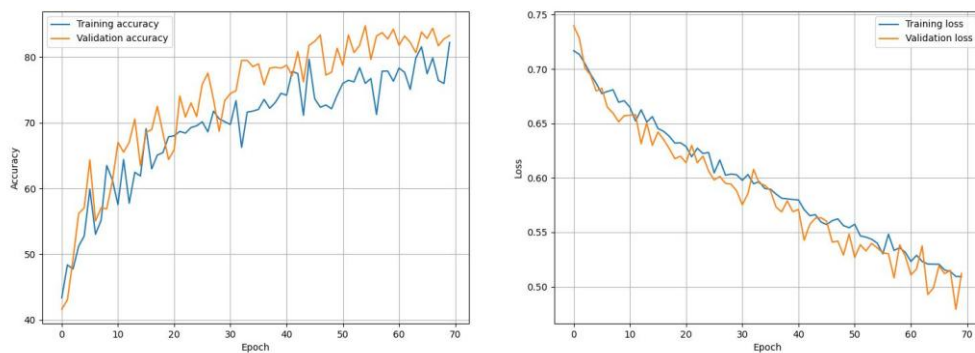


图 25 学习率为 $3e-4$

结果如上所示，学习率为 $1e-3$ 时模型收敛较快但过拟合现象明显；学习率为 $1e-4$ 时模型训练稳定但存在欠拟合问题；而学习率为 $3e-4$ 时模型在验证集上取得了较高且稳定的准确率，训练过程平稳，泛化性能较优。因此，本文最终选取学习率为 $3e-4$ 的模型作为最终实验模型。

5.3 VGG19 模型

对于 VGG 网络，本文采用基于预训练模型权重，固定训练轮次为 100，比较不同学习率的方式进行模型训练。训练结果如图所示。

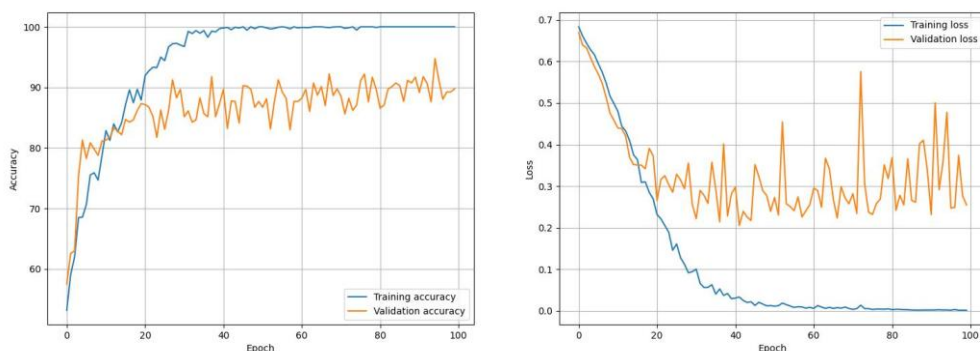


图 26 学习率为 $1e-3$ 结果图

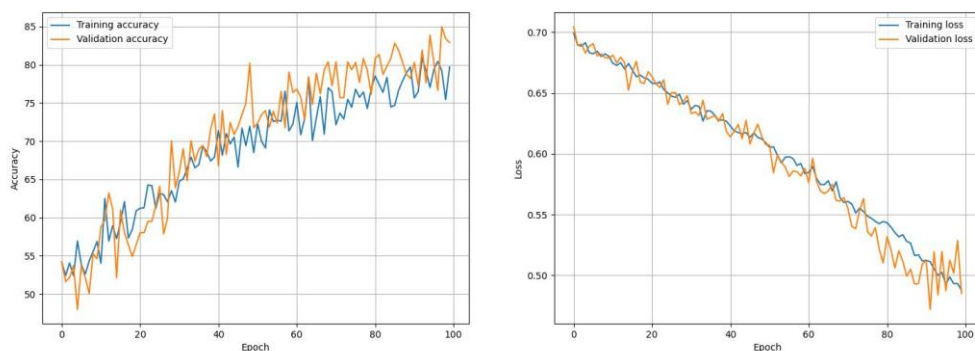


图 27 学习率为 $1e-4$ 结果图

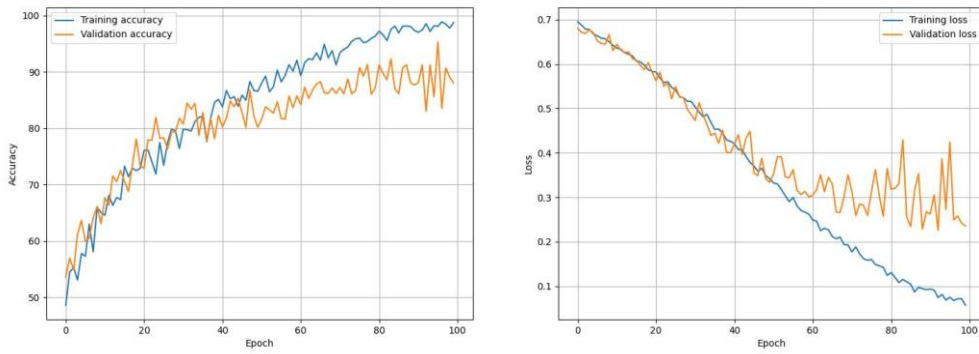


图 28 学习率为 3e-4 结果图

结果如上所示，对于学习率为 $1e-3$ 和 $3e-4$ 的情况下，模型均出现了过拟合，准确率达到 99%，训练集损失不断下降，而验证集损失到后期几乎不变。而在最小的学习率 $1e-4$ 下，模型收敛迅速，且没有出现过拟合现象，验证损失和训练损失都在不断下降，最终准确率为 80% 左右，故选取该模型作为最终模型。

5.3 不同模型性能对比

对三种模型分别在验证集上进行验证，其中验证集为原数据集按照 train/val 的 9: 1 进行划分，计算三种模型的混淆矩阵，同时，引入 fuse model（采用投票机制，如果有两种以上模型的决策为阴性或阳性，该正确结果就为阴性或阳性；反之，采取 VGG 模型的预测结果）。对于的混淆矩阵结果如下所示。

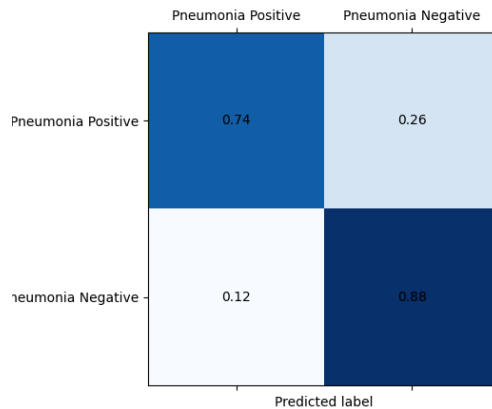


图 29 AlexNet

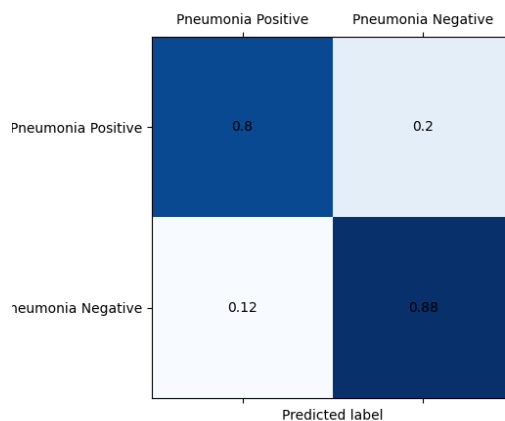


图 30 GoogLeNet

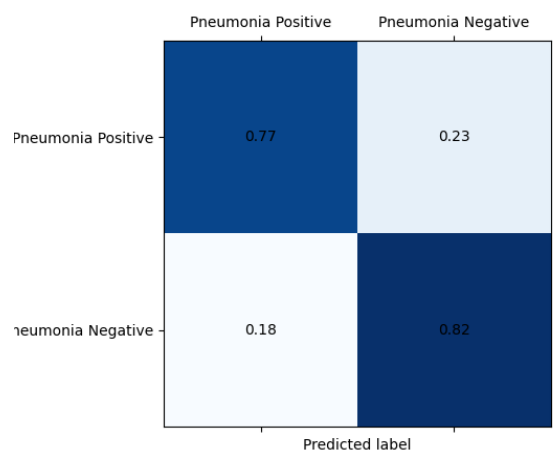


图 31 VGG19

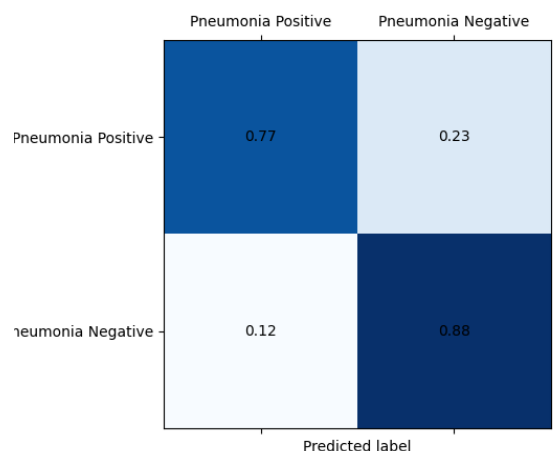


图 32 Fuse Model

由上述混淆矩阵，可得到如下表格。

表格 1 分类效果表

模型	阳性识别率	阴性识别率	主要特点
AlexNet	0.74	0.88	阴性强，阳性漏检多
GoogleNet	0.80	0.88	整体最优，阳性识别最好
VGG19	0.77	0.82	均衡但性能略低
融合模型	0.77	0.88	稳定性最好，泛化更强

● 结论

由上述结果表明，不同模型在肺炎阳性与阴性样本上的识别能力存在差异。**AlexNet** 对阴性样本具有较高识别率，但对阳性样本存在较高漏检率；**GoogleNet** 在两类样本上均表现出较优的识别能力，尤其在肺炎阳性检测方面表现突出；**VGG19** 的整体性能相对均衡，但略逊于 **GoogleNet**。融合模型通过对多个模型预测结果进行决策级融合，在保持较高阴性识别率的同时提高了预测结果的稳定性，综合性能优于单一模型。

第六节 集成应用开发

本文实现了一个简单的肺炎感染影像智能识别应用界面。用户可以通过界面上上传 CT 影像，输入三种模型的最佳 ckpt 路径，系统自动完成影像预处理和模型推理，并输出肺炎感染识别结果，其中融合模型为投票表决：如果有两种以上模型的决策为阴性或阳性，该正确结果就为阴性或阳性；反之，采取 VGG 模型的预测结果。

● 对于阳性图像

随机选取了两张阳性图像，不同模型的预测结果如下所示。可以看到，所有模型的预测均为阳性，结果正确。

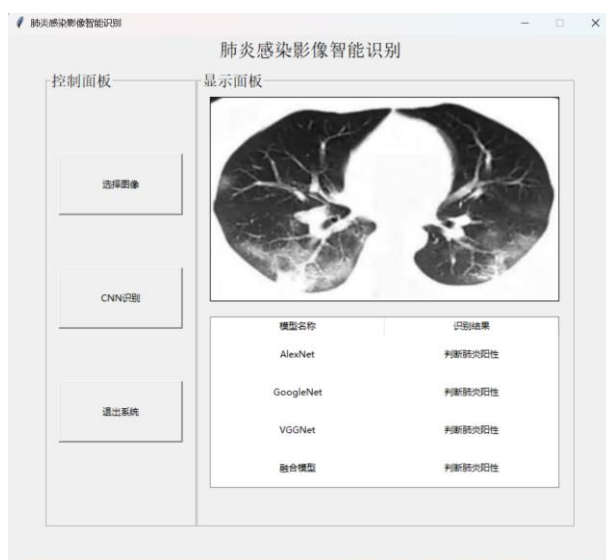


图 33 阳性图像预测



图 34 阳性图像预测

● 对于阴性图像

随机选取了两张阴性图像，不同模型的预测结果如下所示。可以看到，所有模型的预测均为阴性，结果正确。

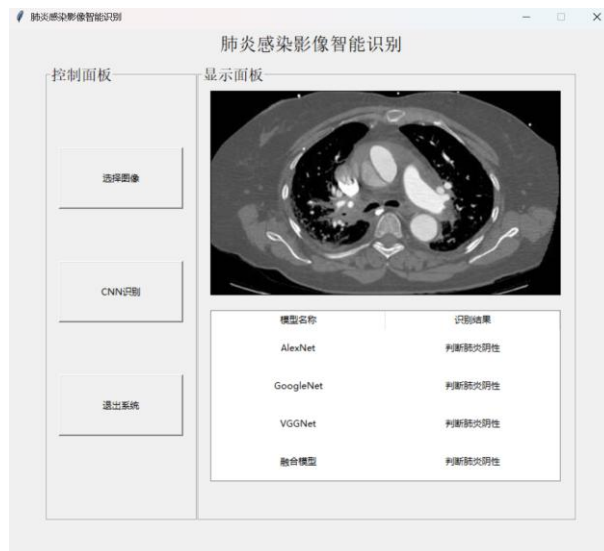


图 35 阴性图像预测



图 36 阴性图像预测

第七节 结论与展望

7.1 结论

- 1) 本文围绕肺炎感染影像的自动识别问题，基于深度学习方法构建了一套完整的肺炎影像二分类识别流程。以肺炎 CT 影像数据为研究对象，选取 AlexNet、GoogLeNet 和 VGG19 三种经典卷积神经网络模型开展实验，对比分析了预训练与非预训练模型、不同学习率以及不同训练轮数对模型性能的影响，并在此基础上实现了一个简单的肺炎感染影像智能识别应用界面。
- 2) 实验结果表明，引入 ImageNet 预训练权重能够显著提升模型的收敛速度和分类性能，尤其在数据规模相对有限的医学影像任务中，迁移学习方法能够有效增强模型的特征提取能力。通过对不同学习率和训练轮数的实验分析发现，合理的超参数设置对于模型性能具有重要影响，其中学习率为 $3e-4$ 、训练轮次为 70 时，模型在收敛速度与泛化性能之间取得了较好的平衡。
- 3) 在不同模型的对比实验中，GoogLeNet 在肺炎阳性与阴性样本的识别能力上均表现较优，尤其

在降低肺炎阳性样本漏检率方面具有明显优势。进一步引入基于投票机制的融合模型后，模型预测结果的稳定性得到了提升，在保持较高阴性识别率的同时，提高了整体泛化能力。实验结果验证了深度学习方法在肺炎感染影像智能识别任务中的有效性和可行性。

7.2 展望

尽管本文所提出的肺炎感染影像智能识别方法在实验中取得了一定效果，但仍存在一些不足，有待在后续工作中进一步完善和提升。

- 1) 在数据层面，本文所使用的数据集规模相对有限，样本分布也较为简单，模型训练容易过拟合。
- 2) 在模型层面，本文主要选用了经典的卷积神经网络结构进行实验。后续研究可进一步引入如 ResNet、DenseNet 以及 Vision Transformer 等网络结构，以探索其在肺炎影像识别任务中的性能提升空间。

综上所述，本文的研究为基于深度学习的肺炎感染影像智能识别提供了一定的实验基础和实践参考，未来在数据、模型和系统应用等方面仍具有较大的研究和改进空间。

参考文献

- [1] Krizhevsky A, Sutskever I, Hinton G E. ImageNet Classification with Deep Convolutional Neural Networks. <https://papers.nips.cc/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf>
- [2] Simonyan K, Zisserman A. Very Deep Convolutional Networks for Large-Scale Image Recognition. <https://arxiv.org/abs/1409.1556>
- [3] Szegedy C, et al. Going Deeper with Convolutions. <https://arxiv.org/abs/1409.4842>
- [4] LeCun Y, Bengio Y, Hinton G. Deep Learning. <https://www.nature.com/articles/nature14539>
- [5] Litjens G, et al. A survey on deep learning in medical image analysis. <https://www.sciencedirect.com/science/article/pii/S1361841517301135>
- [6] Rajpurkar P, et al. CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning. <https://arxiv.org/abs/1711.05225>
- [7] Pan S J, Yang Q. A Survey on Transfer Learning. <https://ieeexplore.ieee.org/document/5288526>
- [8] Shorten C, Khoshgoftaar T M. A survey on image data augmentation for deep learning. <https://journalofbigdata.springeropen.com/articles/10.1186/s40537-019-0197-0>
- [9] He K, et al. Deep Residual Learning for Image Recognition. <https://arxiv.org/abs/1512.03385>
- [10] Zhou Z H. Machine Learning. <https://link.springer.com/book/10.1007/978-981-15-1967-3>

● AlexNet 模型训练节选代码

```
def main():
    epochs = 100
    device = 'cuda' if torch.cuda.is_available() else 'cpu'
    lr = 3e-4
    wd = 1e-5

    train_data, val_data, cls2idx = split_data()

    batch_size = 32
    num_workers = 0

    train_set = DataSet(train_data, cls2idx, transform=transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
    ]))
    val_set = DataSet(val_data, cls2idx, transform=transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
    ]))

    train_loader = DataLoader(train_set, batch_size=batch_size,
                              shuffle=True, num_workers=num_workers)
    val_loader = DataLoader(val_set, batch_size=batch_size,
                             shuffle=True, num_workers=num_workers)

    model = alexnet(weights=AlexNet_Weights.IMAGENET1K_V1)
    # model = alexnet(weights=None)
    model.classifier = nn.Sequential(
        nn.Dropout(p=0.5),
        nn.Linear(256 * 6 * 6, 4096),
        nn.ReLU(inplace=True),
        nn.Dropout(p=0.5),
        nn.Linear(4096, 4096),
        nn.ReLU(inplace=True),
```

```
        nn.Linear(4096, 2),
    )
    model.to(device)

    criterion = nn.CrossEntropyLoss().to(device)
    optimizer = optim.SGD(model.parameters(), lr=lr, weight_decay=wd)
```

省略

● GoogLeNet 模型训练节选代码

```
def main():
    epochs = 70
    device = 'cuda' if torch.cuda.is_available() else 'cpu'
    lr = 1e-3
    wd = 1e-5

    train_data, val_data, cls2idx = split_data()

    batch_size = 32
    num_workers = 0

    train_set = DataSet(train_data, cls2idx, transform=transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
    ]))
    val_set = DataSet(val_data, cls2idx, transform=transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
    ]))

    train_loader = DataLoader(train_set, batch_size=batch_size,
                              shuffle=True, num_workers=num_workers)
    val_loader = DataLoader(val_set, batch_size=batch_size,
                            shuffle=True, num_workers=num_workers)
```

```
model = googlenet(weights=GoogLeNet_Weights.IMAGENET1K_V1)
model.fc = nn.Linear(1024, 2)
model.to(device)

criterion = nn.CrossEntropyLoss().to(device)
optimizer = optim.SGD(model.parameters(), lr=lr, weight_decay=wd)
```

- **VGG 模型训练节选代码**

```
def main():
    epochs = 70
    device = 'cuda' if torch.cuda.is_available() else 'cpu'
    lr = 1e-4
    wd = 1e-5

    train_data, val_data, cls2idx = split_data()

    batch_size = 32
    num_workers = 0

    train_set = DataSet(train_data, cls2idx, transform=transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
    ]))
    val_set = DataSet(val_data, cls2idx, transform=transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
    ]))

    train_loader = DataLoader(train_set, batch_size=batch_size,
                              shuffle=True, num_workers=num_workers)
    val_loader = DataLoader(val_set, batch_size=batch_size,
                             shuffle=True, num_workers=num_workers)

    model = vgg19(weights=VGG19_Weights.IMAGENET1K_V1)
    model.classifier = nn.Sequential(
```



```

        nn.Linear(512 * 7 * 7, 4096),
        nn.ReLU(True),
        nn.Dropout(p=0.5),
        nn.Linear(4096, 4096),
        nn.ReLU(True),
        nn.Dropout(p=0.5),
        nn.Linear(4096, 2),
    )
    model.to(device)

    criterion = nn.CrossEntropyLoss().to(device)
    optimizer = optim.SGD(model.parameters(), lr=lr, weight_decay=wd)

```

- 模型评估代码

```

def eval(model):
    model.eval()

    targets = []
    preds = []

    with torch.no_grad():
        for i, (images, labels) in enumerate(val_loader):
            images = images.to(device)
            outputs = model(images)

            labels = labels.detach().cpu().numpy().tolist()

            _, predicted = torch.max(outputs.data, 1)
            predicted = predicted.detach().cpu().numpy().tolist()

            targets.extend(labels)
            preds.extend(predicted)

    correct = np.sum(np.array(targets) == np.array(preds))
    acc = correct / len(targets)
    print(acc)

    m = confusion_matrix(targets, preds, normalize='true')

```

```

fig, ax = plt.subplots()
ax.matshow(m, cmap='Blues')
for i in range(len(m)):
    for j in range(len(m)):
        plt.annotate(round(m[j, i], 2), xy=(
            i, j), horizontalalignment='center', verticalalignment='center')

ax.set_ylabel('True label')
ax.set_xlabel('Predicted label')
ax.set_xticks([0, 1], labels=['Pneumonia Positive',
                               'Pneumonia Negative'], fontsize=10)
ax.set_yticks([0, 1], labels=['Pneumonia Positive',
                               'Pneumonia Negative'], fontsize=10)

# 评估 alexnet
alex = alexnet()
alex.classifier = nn.Sequential(
    nn.Dropout(p=0.5),
    nn.Linear(256 * 6 * 6, 4096),
    nn.ReLU(inplace=True),
    nn.Dropout(p=0.5),
    nn.Linear(4096, 4096),
    nn.ReLU(inplace=True),
    nn.Linear(4096, 2),
)
alex.eval()
alex.load_state_dict(torch.load(
    '/data1/wph/final/cv-big-homework/alex/alex_70_0.0003.pth'))
alex.to(device)

eval(alex)
plt.savefig("alex/confusion_matrix.png")
plt.show()
##### 其他模型实现雷同，省略

```

● UI 页面代码

```

import torch
from torch import nn
from torchvision.models import alexnet, googlenet, vgg19

```

```

from torchvision.transforms import transforms

import tkinter as tk
from PIL import Image, ImageTk
from tkinter import messagebox as msg
from tkinter import filedialog as filedialog
from tkinter import ttk

class Window:
    def __init__(self, w=800, h=700):
        # 主窗口
        self.window = tk.Tk(className='肺炎感染影像智能识别')
        self.window.geometry(f'{w}x{h}')
        self.window.resizable(0, 0)

        # 标题
        self.title = tk.Label(self.window, text='肺炎感染影像智能识别', font=("", 18))
        self.title.pack(pady=5)

        self.init_frame()

        # 加载模型
        self.alex = alexnet()
        self.alex.classifier = nn.Sequential(
            nn.Dropout(p=0.5),
            nn.Linear(256 * 6 * 6, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(p=0.5),
            nn.Linear(4096, 4096),
            nn.ReLU(inplace=True),
            nn.Linear(4096, 2),
        )
        self.alex.eval()
        self.alex.load_state_dict(torch.load('alex/alex.pth'))

        self.google = googlenet(aux_logits=False, transform_input=True)
        self.google.fc = nn.Linear(1024, 2)

```

```

self.google.eval()
self.google.load_state_dict(torch.load('google/google.pth'))

self.vgg = vgg19()
self.vgg.classifier = nn.Sequential(
    nn.Linear(512 * 7 * 7, 4096),
    nn.ReLU(True),
    nn.Dropout(p=0.5),
    nn.Linear(4096, 4096),
    nn.ReLU(True),
    nn.Dropout(p=0.5),
    nn.Linear(4096, 2),
)
self.vgg.eval()
self.vgg.load_state_dict(torch.load('vgg/vgg.pth'))

self.window.mainloop()

def init_frame(self):
    # 控制面板
    self.init_control_frame()
    # 显示区域
    self.init_show_frame()

    self.image = None

def init_control_frame(self):
    """添加控制面板"""
    self.control_frame = tk.LabelFrame(self.window, text='控制面板', width=200,
height=600, font=("", 15))
    self.control_frame.place(x=50, y=50)

    self.bt_choice_im = tk.Button(self.control_frame, text='选择图像', width=22, height=4,
command=self.choice_image)
    self.bt_choice_im.place(x=15, y=85)

    self.bt_cnn = tk.Button(self.control_frame, text='CNN 识别', width=22, height=4,

```

```

command=self.cnn)

        self.bt_cnn.place(x=15, y=235)

        self.bt_exit = tk.Button(self.control_frame, text='退出系统', width=22, height=4,
command=self.exit_sys)

        self.bt_exit.place(x=15, y=385)

    def init_show_frame(self):
        """添加显示区域窗口"""

        self.show_frame = tk.LabelFrame(self.window, text='显示面板', width=500, height=600,
font=("", 15))

        self.show_frame.place(x=250, y=50)

        self.imageTk = ImageTk.PhotoImage(Image.new("RGB", (int(384 * 1.2), int(224 * 1.2)),
(179, 199, 255)))

        self.image_label = tk.Label(self.show_frame, image=self.imageTk, relief="solid",
borderwidth=1)

        self.image_label.place(x=15, y=10)

        self.table_style = ttk.Style()
        self.table_style.configure("Treeview", rowheight=50)

        self.table = ttk.Treeview(self.show_frame, show='headings', height=4)
        self.table["columns"] = ("模型名称", "识别结果")
        self.table.column("模型名称", width=230, anchor='center')
        self.table.column("识别结果", width=230, anchor='center')
        self.table.heading("模型名称", text="模型名称", anchor='center')
        self.table.heading("识别结果", text="识别结果", anchor='center')
        self.table.place(x=15, y=300)

        self.table.insert("", 0, values=("AlexNet", ""))
        self.table.insert("", 1, values=("GoogleNet", ""))
        self.table.insert("", 2, values=("VGGNet", ""))
        self.table.insert("", 3, values=("融合模型", ""))

    def choice_image(self):
        # 初始化工作

        self.init_frame()

```

```

# 选择文件
file_path = filedialog.askopenfilename(initialdir='./')

if file_path:
    if file_path.split('.')[-1] not in {"jpg", "png"}:
        msg.showerror(message='文件格式不支持')
    else:
        # 加载图像
        image = Image.open(file_path).convert('RGB')
        self.image = image.copy()

        self.image_tk = ImageTk.PhotoImage(image.resize((self.image_tk.width(),
self.image_tk.height()))))
        self.image_label['image'] = self.image_tk

def preprocess(self, image: Image.Image):
    trans = transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
    ])
    image = trans(image)
    return torch.unsqueeze(image, 0)

def pred2idx(self, pred):
    _, predicted = torch.max(pred.data, 1)
    predicted = predicted.detach().cpu().numpy().tolist()
    return predicted[0]

def cnn(self):
    image = self.preprocess(self.image)
    alex_pred = self.alex(image)
    alex_pred = self.pred2idx(alex_pred)

    google_pred = self.google(image)
    google_pred = self.pred2idx(google_pred)

```

```

vgg_pred = self.google(image)
vgg_pred = self.pred2idx(vgg_pred)

fuse_pred = 1 if sum([alex_pred, google_pred, vgg_pred]) >= 2 else 0
if fuse_pred == 0:
    fuse_pred = vgg_pred

id2cls = {0: "肺炎阳性", 1: "肺炎阴性"}

alex_pred = id2cls[alex_pred]
google_pred = id2cls[google_pred]
vgg_pred = id2cls[vgg_pred]
fuse_pred = id2cls[fuse_pred]

self.table.insert("", 0, values=("AlexNet", f"判断{alex_pred}"))
self.table.insert("", 1, values=("GoogleNet", f"判断{google_pred}"))
self.table.insert("", 2, values=("VGGNet", f"判断{vgg_pred}"))
self.table.insert("", 3, values=("融合模型", f"判断{fuse_pred}"))

def exit_sys(self):
    if msg.askyesno('提示', '是否退出系统'):
        exit()

w = Window()

```